

FINTECH MULTI-AGENT RAG + VOICE

Architecture Decision Record

Pre-Build Stack Decisions · Phase 0 Baseline

Version 1.1 · 04 April 2026 · Living document — updated per phase

Purpose: This document captures every architecture decision made before Phase 1 engineering begins. For each decision it records: the question that prompted it, the options considered, the decision taken, and the precise rationale. It will be updated at the end of each phase as new decisions are made.

Document structure

Section	Contents
1. System overview	What is being built, hard constraints, design philosophy
2. ADR-001 LLM inference	Groq vs Ollama — speed, cost, data residency split
3. ADR-002 STT	Groq Whisper vs Deepgram vs Whisper.cpp — chosen: Groq Whisper
4. ADR-003 TTS	Cartesia Sonic vs Deepgram Aura vs Coqui — chosen: Cartesia
5. ADR-004 Chunking & retrieval	Chunk sizes, top-k, voice vs text tuning
6. ADR-005 Evaluation framework	RAGAS gaps, DeepEval, custom harness, golden dataset
7. ADR-006 Mobile client	LiveKit SDK options — chosen: React Native + Expo
8. ADR-007 RAG orchestration	LlamaIndex core — HierarchicalNodeParser, AutoMerging, SubQuestion, Citation engines
9. Latency budget	Confirmed numbers per layer with chosen stack
10. Locked stack reference	Complete tool list — every decision in one table
11. Phase 1 task breakdown	9-task build order for RAG pipeline with Claude Code on VS Code
12. Open items	Decisions deferred to later phases

1. System overview

What is being built

A fintech compliance voice assistant that bank employees can speak to or type queries into. It covers four domain workflows: regulatory Q&A (RBI/SEBI/FEMA), customer support, loan/credit eligibility assessment, and KYC/AML document processing. The entire inference stack runs on open-source models. No paid LLM API is required for production — Groq is used for speed during development and demonstration.

Hard constraints — non-negotiable

Constraint	Requirement	Implication
Data residency	Customer PII never leaves the server	KYC/AML + Credit must use Ollama local inference, not Groq
Open-source stack	No paid LLM APIs in production	Mistral-7B via Ollama for PII-sensitive workflows
Auditability	Every query, source, and decision logged	Append-only audit JSONL + Langfuse traces
Accuracy over recall	Wrong regulatory answer is worse than no answer	NLI hallucination gate + citation grounding + human escalation
Voice latency	Sub-600ms to first spoken word	Groq STT + Groq LLM + Cartesia TTS all streamed
Open test data	No proprietary datasets for evaluation	RBI.org.in + SEBI.gov.in + BIS.org public PDFs

Design philosophy

- **Cloud LLMs for public documents only.** Regulatory circulars (RBI, SEBI, FEMA) are public. Claude/Groq is acceptable. Credit risk models and KYC data are proprietary + PII. Ollama only.
- **Stream everything in the voice path.** STT streams chunks as user speaks. LLM streams tokens as generated. TTS speaks first word before full answer is ready. Latency is perceived not measured — users hear progress immediately.
- **One Groq API key serves STT + LLM.** Groq Whisper-large-v3 and Groq Llama-3.1-8B both run on Groq's LPU. Both are on the free tier. No second account needed.
- **Pipecat orchestrates, LiveKit transports.** These are distinct responsibilities. LiveKit = WebRTC audio plumbing. Pipecat = processor chain (VAD → STT → RAG → TTS). The RAG pipeline plugs into Pipecat as a custom LLM processor.

ADR-001	LLM Inference Layer	Status: DECIDED
---------	---------------------	-----------------

WHY	Questions that drove this decision: "Can we use Groq's inference with Llama — I have the key for that." "Can we reduce latency below 3 seconds for voice?" "What is the right model per domain — 8B vs 70B vs Mixtral?"
-----	---

Context

The original plan used Ollama Mistral-7B for all inference (fully local, no API dependency). The constraint was latency — Mistral-7B on CPU takes 800ms TTFT, on a mid-range GPU 200ms. For a voice assistant, 800ms for just the LLM step makes the 3-second total target impossible. The user confirmed a Groq API key is available, which changes the calculation entirely.

Options evaluated

Option	TTFT	Tokens/sec	Cost	Data leaves server?
Ollama Mistral-7B (CPU)	~800ms	~25 t/s	Free	No
Ollama Mistral-7B (GPU)	~200ms	~80 t/s	Free	No
Groq Llama-3.1-8B-Instant	~150ms	~750 t/s	Free tier	Yes (non-PII only)
Groq Llama-3.3-70B	~300ms	~280 t/s	Free tier	Yes (non-PII only)
Groq Mixtral-8x7B	~200ms	~480 t/s	Free tier	Yes (non-PII only)

Decision

DECISION	Split inference by domain and data sensitivity: Voice queries + Customer support → Groq Llama-3.1-8B-Instant (150ms TTFT) Complex regulatory Q&A; (text) → Groq Llama-3.3-70B (300ms, best accuracy) Credit decisioning (structured JSON) → Groq Mixtral-8x7B (reliable JSON output) KYC/AML workflows (PII data) → Ollama Mistral-7B local (PII never leaves server)
----------	---

Rationale

- Groq's LPU architecture gives 5x the tokens/sec of any GPU cloud at zero cost on the free tier. For voice, 150ms TTFT from Groq vs 800ms from local CPU is the difference between a good UX and a broken one.
- The data residency constraint is respected: Regulatory circulars (RBI, SEBI, FEMA) are public documents — sending them to Groq is acceptable. KYC data contains Aadhaar numbers, PAN cards, customer names — these must never leave the server. Ollama handles this.
- The free tier limit (6,000 TPM, 14,400 RPD) is sufficient for development and demonstration. One RAG call uses ~1,500 tokens, giving approximately 4 RAG calls per minute. Rate limit headers must be read and backoff implemented before production.
- Mixtral-8x7B is chosen for credit JSON because its mixture-of-experts architecture has been empirically more reliable at following strict JSON schemas than comparably-sized dense models.

CAUTION	The 6,000 TPM free tier limit means max 4 concurrent RAG calls per minute. Build token backoff into the FastAPI gateway from day one. Read x-ratelimit-remaining-tokens headers on every Groq response.
---------	---

ADR-002	Speech-to-Text (STT)	Status: DECIDED
---------	----------------------	-----------------

WHY	Questions that drove this decision: "Can we run Whisper on Groq free?" "Deepgram vs Cartesia — is it about STT or TTS?" "Does LiveKit convert PCM to 16kHz WAV or does the microphone capture it that way?"
-----	---

Context

Three credible STT options exist. The choice affects latency, accuracy on financial domain terminology (CIBIL, PMLA, FEMA, Aadhaar), and operational cost. The user confirmed a Groq API key is available. Groq runs Whisper-large-v3 on its LPU hardware at speeds unavailable anywhere else.

Audio format clarification (resolved during discussion)

The microphone captures 48kHz stereo PCM. WebRTC encodes it as Opus (compressed) for network transport. LiveKit Python SDK on the server decodes Opus back to PCM, then resamples to 16kHz mono. Pipecat receives `AudioRawFrame(sample_rate=16000, num_channels=1)`. No WAV file is created — it is raw PCM bytes. Groq Whisper then receives exactly the format it expects, skipping its internal downsample step.

Options evaluated

Option	Latency	Accuracy (financial terms)	Cost	Streaming
Groq Whisper-large-v3	~100ms	High (Whisper trained on diverse domain text)	Free (2,000 req/day)	Chunked
Deepgram Nova-2	~200ms	Highest (financial model fine-tune)	Paid	True word-by-word
Whisper.cpp (tiny.en, local)	~300ms CPU	Medium (lower on Indian English + acronyms)	Free	No

Decision

DECISION	Groq Whisper-large-v3 via Groq API. Rationale: lowest latency (100ms), zero additional cost (uses existing Groq key), same Whisper accuracy as local Whisper.cpp but 3x faster. Free tier: 2,000 requests/day, 7,200 audio-seconds/hour — sufficient for dev + demo. Min charge: 10 seconds per request — design VAD to send complete utterances, not fragments.
----------	--

Rationale

- Groq Whisper-large-v3 is the same model as running Whisper locally but 3x faster due to LPU hardware. There is no quality tradeoff — only a speed benefit.
- Deepgram Nova-2 has better end-of-utterance detection (VAD) and a financial domain fine-tune. It is the better production choice when accuracy on terms like 'CIBIL score' or 'PMLA section 12' matters. Deferred to production phase — Deepgram can be swapped in via Pipecat config flag.
- Whisper.cpp local is kept as a fallback for offline development when no internet is available.
- The 10-second minimum billing on Groq means the VAD (Silero in Pipecat) must buffer complete utterances. Sending 2-second partial audio clips would burn through the 2,000 req/day limit in minutes.

ADR-003	Text-to-Speech (TTS)	Status: DECIDED
---------	----------------------	-----------------

WHY	Questions that drove this decision: "Deepgram vs Cartesia — which one for TTS?" "I have credits for Cartesia." "Does Groq have a TTS option?"
-----	---

Options evaluated

Option	TTFT (streaming)	Quality	Cost	Pipecat support
Cartesia Sonic	~90ms	Best-in-class, context-aware prosody	Credits available	Native processor
Deepgram Aura	~120ms	Very good	Paid	Native processor
Groq PlayAI Dialog v1.0	Unknown	Good	Paid only (no free tier)	Custom processor needed
Coqui TTS (local)	~400ms	Acceptable	Free	Custom processor needed
Bark (local)	~1,200ms	Good but slow	Free	Custom processor needed

Decision

DECISION	Cartesia Sonic — using existing credits. 90ms TTFT with byte-level streaming: LLM output tokens pipe directly to Cartesia without waiting for the full answer. User hears speech 90ms after first LLM token. Native Pipecat processor: two-line integration, no custom code. Context-aware prosody: handles 'RBI', 'CIBIL', 'FEMA' as abbreviations correctly.
----------	--

Rationale

- Cartesia has the lowest TTFT of any TTS option (90ms). For voice UX, every millisecond of TTS latency is directly felt by the user. 90ms vs 400ms (Coqui) is the difference between natural conversation and a noticeable pause.
- Byte-level streaming is critical: Cartesia can speak 'For NRI customers...' while the LLM is still generating '...the following documents are required'. This reduces perceived latency from full-answer generation time to first-token time.
- Groq TTS (PlayAI) is paid-only — no free tier. Eliminated immediately.
- Deepgram Aura is the fallback: if Cartesia credits run out, swap to Deepgram via a single config change in the Pipecat pipeline.

ADR-004	Chunking Strategy and Retrieval	Status: DECIDED
---------	---------------------------------	-----------------

WHY	Questions that drove this decision: "Can we discuss better model to reduce latency — the chunking and retrieval?" "3 seconds is too much — what is achievable?" "Does top-k size affect voice latency?"
-----	---

Chunking strategy

Three-level hierarchical chunking with sentence-aware boundaries. Documents are split at sentence boundaries but capped at token limits per level. This gives semantic coherence (regulatory clauses are not split mid-sentence) with predictable chunk sizes for cost estimation.

Level	Tokens	Indexed in	Used for	Rationale
L3 — leaf	100 tokens	Weaviate (vector) + OpenSearch (BM25)	Voice queries, precise retrieval	Small = fast embedding + fast context generation
L2 — mid	400 tokens	Docstore only	Text queries, balanced context	Retrieved via auto-merge from L3
L1 — full section	1,500 tokens	Docstore only	Complex sub-questions	Only reached when auto-merge triggers

Why 100 tokens (not 128) for voice

Voice queries are short (5-10 words). The retrieved context must generate a speakable answer of 1-3 sentences. A 128-token leaf may contain a complete answer plus noise. 100 tokens is tighter, gives the LLM less to hallucinate from, and reduces the context window passed to Groq — directly reducing generation time and token consumption.

Top-k tuning for voice vs text

Mode	Top-k	Context tokens to LLM	Groq generation time	When to use
Voice	top-2 + auto-merge	~200-800 tokens	Fastest (~300ms)	All voice queries
Text (simple)	top-3	~300-1,200 tokens	Fast (~500ms)	Support, simple regulatory
Text (complex)	top-4	~400-1,600 tokens	Moderate (~800ms)	Multi-hop regulatory
Avoid	top-6+	~600-2,400 tokens	Slow (1,200ms+)	Marginal quality gain

Retrieval pipeline

DECISION	Hybrid retrieval: BGE-M3 dense (Weaviate) + BM25 sparse (OpenSearch) fused with RRF (k=60). BGE-M3: MIT licensed, runs locally, 1024-dim, handles Indian English + regulatory acronyms. RRF fusion: rank-based, avoids BM25/cosine score scale mismatch, no normalisation needed. BGE cross-encoder reranker: final precision pass after fusion. Auto-merge: if >=50% of a parent's children retrieved, upgrade to parent chunk.
----------	--

ADR-005	Evaluation Framework	Status: DECIDED
---------	----------------------	-----------------

WHY	Questions that drove this decision: "Is RAGAS a golden eval framework for this?" "What does RAGAS miss for fintech + voice?" "What is the LLM-as-judge bias problem with RAGAS?"
-----	--

RAGAS — what it covers and what it misses

RAGAS metric	What it measures	Gap for fintech RAG
Faithfulness	Are answer claims grounded in retrieved context? (LLM-as-judge)	Scores high even if retrieved context is from a superseded 2019 circular
Answer relevance	Does the answer address the question? (embedding similarity)	Does not check if the answer is factually correct by current regulation
Context recall	Did retrieval find relevant docs? (needs ground truth)	Needs golden dataset — cannot self-evaluate
Context precision	Are retrieved docs actually used? (token overlap)	Token overlap misses semantic paraphrasing

RAGAS LLM-as-judge bias — critical issue

CAUTION	If you use Groq Llama-3.1-8B to generate answers AND as the RAGAS judge, the judge will agree with its own outputs. Always use a different model as judge: Mixtral-8x7B or Ollama Mistral-7B (free, local). Never use the generation model as evaluator.
---------	--

Full evaluation stack

Tool	What it adds	Open-source ?	Cost
RAGAS	Faithfulness, relevance, context recall/precision	Yes (Apache 2.0)	Free (needs judge LLM)
DeepEval	Hallucination classification, answer correctness, toxicity	Yes (Apache 2.0)	Free
Promptfoo	Regression testing, red teaming, prompt comparison	Yes (MIT)	Free
JIWER	STT Word Error Rate on domain financial terms	Yes (MIT)	Free
Custom harness	Voice latency (TTFT, full answer), turn-taking accuracy, regulatory accuracy vs golden dataset	Build it	Free

Golden dataset — the most important evaluation asset

DECISION	20 questions minimum, 5 per domain, from public RBI/SEBI/BIS documents. Format per question: {question, ground_truth, reference_doc, reference_section, domain, difficulty} Sources: RBI.org.in Master Directions, SEBI.gov.in circulars, BIS.org Basel III docs Run nightly: RAGAS + DeepEval on all 20 questions. Alert if any metric drops >0.05. LLM judge: Mixtral-8x7B (different from generation model — avoids self-evaluation bias).
----------	---

ADR-006	Mobile Client Platform	Status: DECIDED
---------	------------------------	-----------------

WHY	Questions that drove this decision: "Can I use LiveKit for mobile apps?" "LiveKit is the transport layer right, and Pipecat is the orchestrator?" "Does LiveKit convert raw audio to 16kHz mono WAV or is it captured in that format?" "Noise cancellation, echo cancellation, packet loss — are these WebRTC or LiveKit?"
-----	--

WebRTC vs LiveKit — clarified boundary

Component	Responsibility	Where it runs
WebRTC framework (browser built-in)	Noise suppression, echo cancellation, AGC, NAT traversal (ICE/STUN/TURN), jitter buffer, packet loss concealment, Opus encoding	Browser / mobile OS
LiveKit	Signalling server, STUN/TURN infrastructure, Python SDK that implements WebRTC peer, Opus decode + resample to 16kHz mono	Your server + client SDK
Pipecat	Processor chain: VAD → STT → RAG → TTS. Receives 16kHz mono from LiveKit. Your RAG pipeline is the LLM processor.	Your server (Python process)

Audio format journey (resolved question)

Microphone → 48kHz stereo PCM. WebRTC → Opus encoded (compressed, 48kHz internally, ~20ms packets). Network → Opus packets travel to server. LiveKit Python SDK → decode Opus → 48kHz stereo PCM → resample → 16kHz mono PCM. Pipecat → receives AudioRawFrame(sample_rate=16000, num_channels=1). Groq Whisper → receives 16kHz mono, skips its own downsample step.

Mobile SDK options

SDK	Platform	Starter app available	Maturity
Swift SDK	iOS, macOS, visionOS	SwiftUI voice agent starter	Production-ready
Kotlin SDK	Android	Jetpack Compose voice agent starter	Production-ready
Flutter SDK	iOS + Android (one codebase)	Flutter voice agent starter	Production-ready
React Native + Expo	iOS + Android (one JS codebase)	React Native voice agent starter (includes audio visualizer + session management)	Production-ready
JS SDK	Browser	Next.js starter	Production-ready

Decision

DECISION	React Native + Expo for mobile client. Single JavaScript codebase runs on iOS and Android. LiveKit React Native voice agent starter includes: audio visualizer, session management, connect/disconnect controls — no boilerplate to write. Expo simplifies native build process — no Xcode config or Gradle complexity upfront. Server-side Pipecat pipeline is identical regardless of which client SDK is used.
CAUTION	iOS Simulator cannot publish microphone tracks. Must test on real iPhone from day one. Android background: need foreground service (persistent notification) to stay connected when app is backgrounded — use @supersami/rn-foreground-service (included in starter app).

ADR-007	RAG Orchestration Framework — LlamaIndex Core	Status: DECIDED
---------	--	-----------------

WHY	Questions that drove this decision: "Can we use LlamaIndex core library for parsing and chunking?" "It provides auto-merging and retrieval, citations, sub-question decomposition — free?" "Should I use LlamaIndex for everything or just as the query layer?" "I am new to RAG — what does each LlamaIndex component actually do?"
-----	--

Context

Building hierarchical chunking, auto-merging retrieval, sub-question decomposition, and citation tracking from scratch would take 2-3 weeks and produce worse results than LlamaIndex's battle-tested implementations. LlamaIndex core is MIT-licensed, free, and provides exactly the four features this project requires as first-class primitives. The decision is whether to use LlamaIndex for everything (including PDF parsing) or only from the chunking step onward.

The parsing boundary decision

Option	PDF Parser	Chunking onward	Problem
A — LlamaIndex for everything	SimpleDirectoryReader + PDFReader (uses pypdf internally)	LlamaIndex	pypdf fails on multi-column RBI circulars, flattens tables, includes page headers in chunks
B — pdfplumber + LlamaIndex (chosen)	pdfplumber: understands column layout, extracts tables, crops headers/footers	LlamaIndex	None — clean separation of concerns. 5 lines of glue code to wrap text in Document object.

What each LlamaIndex component does

Component	What it does	What you would build without it
HierarchicalNodeParser	Takes a Document, produces 3-level node tree: leaf nodes (100 tok), mid nodes (400 tok), parent nodes (1500 tok). Parent-child relationships wired automatically.	Custom recursive text splitter + parent ID tracking + docstore management. ~150 lines.
AutoMergingRetriever	After dense retrieval returns leaf nodes, checks: if $\geq 50\%$ of a parent's children were retrieved, replace all children with the parent node. Gives richer context without noise.	Custom post-retrieval merge logic with sibling counting. ~80 lines. Easy to get wrong.
SubQuestionQueryEngine	Decomposes complex queries ("Compare 2023 vs 2024 NRI KYC rules") into atomic sub-questions, retrieves for each independently, synthesises into one cited answer.	LLM call to decompose + parallel retrieval per sub-question + merging logic. ~200 lines.
CitationQueryEngine	Adds [SOURCE 1], [SOURCE 2] inline citations in every answer. Returns source_nodes on response object so you always know which chunk grounded which claim.	Custom prompt engineering + response parser to extract and verify citation numbers. ~100 lines.
QueryFusionRetriever	Runs query against multiple retrievers (dense + sparse), fuses results with RRF (built-in mode). Single class, one config.	Custom RRF implementation: rank lists, $1/(k+rank)$ scoring, deduplication. ~60 lines.
SentenceTransformerReranker	Cross-encoder reranker. Takes top-N fused candidates, scores each query+chunk pair jointly, returns top-K. Plugs into any retriever pipeline.	Direct transformers cross-encoder call + sorting + threshold filtering. ~40 lines.

Decision

DECISION

Use LlamaIndex core as the RAG orchestration layer for all steps after parsing. PDF parsing: pdfplumber (handles RBI multi-column layout + tables) Text → Document: LlamaIndex Document(text=..., metadata={...}) Chunking: HierarchicalNodeParser (100 / 400 / 1500 token levels) Embedding: HuggingFaceEmbedding with BGE-M3 Vector indexing: VectorStoreIndex with QdrantVectorStore Keyword indexing: BM25Retriever (LlamaIndex built-in, no OpenSearch needed in dev) Hybrid fusion: QueryFusionRetriever with mode=RECIPROCAL_RANK Auto-merge: AutoMergingRetriever with merge_threshold=0.5 Reranking: SentenceTransformerRerank with BGE cross-encoder Sub-question: SubQuestionQueryEngine (regulatory complex queries) Citations: CitationQueryEngine (all domains) LLM: Groq via llama-index-llms-groq

Install — all free, all open-source

```
pip install llama-index-core llama-index-embeddings-huggingface llama-index-vector-stores-qdrant llama-index-llms-groq llama-index-retrievers-bm25 llama-index-postprocessor-flag-embedding-reranker pdfplumber qdrant-client
```

BM25 vs OpenSearch — development simplification

LlamaIndex's built-in BM25Retriever uses rank-bm25 (pure Python, no Docker needed) for development and testing. This eliminates the OpenSearch container dependency during Phase 1. OpenSearch replaces it in Phase 2 when production-grade keyword search with synonym expansion (KYC <-> Know Your Customer) is needed. The QueryFusionRetriever interface is identical in both cases — swap one line.

CAUTION

LlamaIndex's HierarchicalNodeParser requires the StorageContext docstore to hold parent nodes — they are NOT indexed in the vector store. Only leaf nodes go into Qdrant. Parent nodes live in the SimpleDocumentStore (local dict or Redis). AutoMergingRetriever looks up parents from the docstore at query time. Do not try to index parent nodes in Qdrant — this breaks auto-merging.

11. Phase 1 — RAG pipeline build plan (Claude Code on VS Code)

Build order: complete each task and verify its output before starting the next. Each task produces something printable and testable. Voice pipeline (Pipecat + LiveKit) is deferred — the RAG pipeline is built and tested via Python script first, voice interface added after Phase 1 completes.

How to use this document with Claude Code

Open this project in VS Code with Claude Code. At the start of each session provide this ADR PDF and say: 'Read fintech_rag_adr_v1.1.1.pdf. We are building Phase 1 Task N today. Follow the task specification exactly.' Claude Code reads the actual files, runs the code, sees errors, and iterates on the real codebase. Commit after each task completes.

Project folder structure to create before Task 1

```
fintech_rag/ data/raw/ ← downloaded RBI/SEBI PDFs go here data/processed/ ← parsed text output
from Task 1 src/ ingestion/ ← Tasks 1-3: parser, chunker, metadata retrieval/ ← Tasks 4-8:
embeddings, stores, fusion, reranker generation/ ← Task 9: LLM chain, citation engine evaluation/
← Phase 1 eval: RAGAS, golden dataset tests/ requirements.txt .env ← GROQ_API_KEY, QDRANT_URL
```

Task 1	PDF loading — pdfplumber parser
Concept	What a document is in RAG: raw bytes → parsed text → chunks → embeddings → vectors. The 5 forms data takes. Why you cannot embed a full 200-page PDF.
What to build	Write src/ingestion/parser.py using pdfplumber. Extract text page by page. Strip page headers/footers using crop(). Handle empty pages gracefully. Return plain text string.
Test data	Download: RBI Master Direction on KYC (rbi.org.in, search 'KYC Master Direction'). Save to data/raw/rbi_kyc_master_direction.pdf
Verify it works	Print first 500 characters of extracted text. Verify: no page numbers, no 'RBI Page X of Y' headers, readable regulatory text.
Output	src/ingestion/parser.py — load_pdf(path) -> str
Claude Code prompt	Write src/ingestion/parser.py. Use pdfplumber. Strip headers (top 40px) and footers (bottom 40px) using page.crop(). Handle empty/scanned pages. Test against data/raw/rbi_kyc_master_direction.pdf. Print first 500 chars of output.

Task 2	Chunking — LlamaIndex HierarchicalNodeParser
Concept	Why chunk size matters: too large = irrelevant context + slow generation. Too small = no context = LLM hallucinates. Hierarchical = precise retrieval (leaf) + rich context (parent) in same index.
What to build	Write src/ingestion/chunker.py. Wrap parsed text in LlamaIndex Document. Configure HierarchicalNodeParser: leaf=100 tok, mid=400 tok, parent=1500 tok. Store parents in SimpleDocumentStore, return leaf nodes for indexing.
Test data	Same RBI KYC PDF from Task 1.
Verify it works	Print: total leaf node count, first leaf node text, its parent node text. Verify parent contains the full section that the leaf came from.
Output	src/ingestion/chunker.py — chunk_document(doc) -> (leaf_nodes, docstore)
Claude Code prompt	Write src/ingestion/chunker.py using LlamaIndex HierarchicalNodeParser. Chunk sizes: leaf=100 tokens, mid=400 tokens, parent=1500 tokens. Use get_leaf_nodes() to separate indexable nodes from parent-only nodes. Print count of leaf nodes and text of node[0] and its parent. Use SimpleDocumentStore for parent storage.

Task 3	Metadata — enrich every chunk with searchable fields
Concept	Why metadata is not optional: <code>is_stale</code> drives the staleness guardrail. <code>clearance_level</code> drives ABAC filtering. <code>jurisdiction</code> drives citation display. Without metadata, guardrails in Phase 5 cannot function.
What to build	Write <code>src/ingestion/metadata.py</code> . Extract from filename: <code>jurisdiction</code> (RBI/SEBI/FEMA), <code>doc_type</code> (<code>master_direction/circular/guideline</code>). Extract from text: <code>effective_date</code> (regex on 'dated', 'issued on'). Set defaults: <code>is_stale=False</code> , <code>clearance_level=1</code> , <code>citation_priority=1</code> .
Test data	Same RBI KYC PDF. Also download one SEBI circular to test jurisdiction detection.
Verify it works	Print metadata dict for 3 different chunks. Verify <code>jurisdiction='RBI'</code> , <code>doc_type='master_direction'</code> , <code>effective_date</code> is not <code>None</code> .
Output	<code>src/ingestion/metadata.py</code> — <code>extract_metadata(filename, text) -> dict</code>
Claude Code prompt	Write <code>src/ingestion/metadata.py</code> . Extract <code>jurisdiction</code> from filename pattern (<code>rbi_*</code> , <code>sebi_*</code> , <code>fema_*</code>). Extract <code>doc_type</code> from filename (<code>master_direction</code> , <code>circular</code> , <code>guideline</code>). Extract <code>effective_date</code> using regex on common RBI date patterns. Set: <code>is_stale=False</code> , <code>clearance_level=1</code> , <code>citation_priority=1</code> . Print metadata for the KYC document.

Task 4	Embeddings — BGE-M3 via LlamaIndex HuggingFaceEmbedding
Concept	What an embedding is: a list of 1024 numbers representing meaning. Similar meaning = similar numbers = small cosine distance. This is how the vector store finds relevant chunks without keyword matching.
What to build	Write <code>src/retrieval/embedder.py</code> . Configure <code>HuggingFaceEmbedding</code> with <code>BAAI/bge-m3</code> . Embed 5 chunks. Print shape. Compute cosine similarity between 'KYC documents for NRI' and 3 chunks — verify most relevant ranks highest.
Test data	Use 5 leaf nodes from Task 2 output.
Verify it works	<code>embedding.shape == (1024,)</code> . Similarity between KYC query and KYC chunk > similarity with unrelated chunk.
Output	<code>src/retrieval/embedder.py</code> — <code>get_embed_model()</code> -> <code>HuggingFaceEmbedding</code>
Claude Code prompt	Write <code>src/retrieval/embedder.py</code> . Use <code>HuggingFaceEmbedding(model_name='BAAI/bge-m3')</code> . Embed these 3 texts: 'KYC documents for NRI account opening', 'NRI customers must provide valid passport', 'Housing loan LTV ratio guidelines'. Compute cosine similarity between query and each chunk. Print results. Verify NRI chunk scores higher than housing loan chunk for NRI query.

Task 5	Vector store — index chunks into Qdrant
Concept	HNSW index: a graph structure that finds approximate nearest neighbours in milliseconds. Why Qdrant not FAISS: Qdrant persists to disk, supports metadata filtering, runs as a service. FAISS is in-memory only — fine for research, not production.
What to build	Run Qdrant in Docker (<code>docker run -p 6333:6333 qdrant/qdrant</code>). Write <code>src/retrieval/vector_store.py</code> . Build <code>VectorStoreIndex</code> from leaf nodes using BGE-M3 + <code>QdrantVectorStore</code> . Query: 'What documents does an NRI need for KYC?' — print top 3 results.
Test data	All leaf nodes from the RBI KYC Master Direction (Task 2 output).
Verify it works	Top result text contains NRI + passport/identity document content. Response time < 200ms. Collection visible in Qdrant dashboard at <code>localhost:6333</code> .
Output	<code>src/retrieval/vector_store.py</code> — <code>build_index(nodes, docstore) -> VectorStoreIndex</code>

Claude Code prompt	Write src/retrieval/vector_store.py. Use QdrantVectorStore (localhost:6333, collection='fintech_rag'). Build VectorStoreIndex from leaf nodes with BGE-M3 embeddings. Query 'What documents does an NRI need for KYC?' with similarity_top_k=5. Print each result's text (first 200 chars) and score.
---------------------------	---

Task 6	BM25 keyword retrieval — LlamaIndex BM25Retriever
Concept	BM25 scores documents by term frequency weighted by document length. It finds exact matches that semantic search misses — regulation codes, section numbers, acronyms. 'Section 4.2' scores 0 in Qdrant, high in BM25.
What to build	Write src/retrieval/bm25_store.py using LlamaIndex BM25Retriever. Index same leaf nodes. Query 'Section 4.2 NRI individual customer'. Compare results to Qdrant query — show the difference clearly.
Test data	Same leaf nodes as Task 5.
Verify it works	BM25 result for 'Section 4.2' contains the actual Section 4.2 text. Qdrant result for same query may return a semantically similar but different section.
Output	src/retrieval/bm25_store.py — build_bm25_retriever(nodes) -> BM25Retriever
Claude Code prompt	Write src/retrieval/bm25_store.py. Use LlamaIndex BM25Retriever(nodes=leaf_nodes, similarity_top_k=5). Query 1: 'What documents does an NRI need for KYC?' Query 2: 'Section 4.2 individual customer verification'. Print BM25 results for both queries. Then print Qdrant results for the same queries side by side. Show the difference in what each retriever finds.

Task 7	Hybrid fusion — QueryFusionRetriever with RRF
Concept	RRF (Reciprocal Rank Fusion): score = 1/(k+rank). k=60 is standard. Uses only rank positions — not raw scores. Avoids the BM25/cosine scale incompatibility problem. Documents appearing in both lists get boosted.
What to build	Write src/retrieval/hybrid_retriever.py. QueryFusionRetriever([qdrant_retriever, bm25_retriever], mode=RECIPROCAL_RANK, num_queries=1). Query same questions as Tasks 5+6. Show fused results beat both individual retrievers.
Test data	Same leaf nodes. Use the query where BM25 found something Qdrant missed.
Verify it works	Fused result for 'Section 4.2 NRI KYC' includes both the exact match (from BM25) AND the semantically related chunk (from Qdrant). Neither retriever alone returned both.
Output	src/retrieval/hybrid_retriever.py — build_hybrid_retriever(index, nodes) -> QueryFusionRetriever
Claude Code prompt	Write src/retrieval/hybrid_retriever.py. Use QueryFusionRetriever with mode=RECIPROCAL_RANK, num_queries=1, k=60. Combine Qdrant retriever (top_k=10) and BM25 retriever (top_k=10). Query: 'Section 4.2 NRI individual customer KYC documents required'. Print: BM25-only results, Qdrant-only results, fused results. Show the fused list contains relevant results from both.

Task 8	Reranker + AutoMerging — final precision pass
Concept	Cross-encoder reranker: scores query+chunk as a PAIR (not independently). More accurate than cosine similarity alone. AutoMerging: if 2 of 3 sibling leaves retrieved (>=50%), replace with parent for fuller context.
What to build	Write src/retrieval/pipeline.py combining: QueryFusionRetriever (top-10) → AutoMergingRetriever → SentenceTransformerRerank (top-3). This is the complete retrieval pipeline. One function: retrieve(query) -> List[NodeWithScore].
Test data	Same KYC document. Use query that exercises auto-merge.
Verify it works	Top-3 results are reranked correctly. At least one result is a parent node (longer text) showing auto-merge fired. Total pipeline time < 800ms.

Output	src/retrieval/pipeline.py — retrieve(query, index, docstore) -> List[NodeWithScore]
Claude Code prompt	Write src/retrieval/pipeline.py. Chain: QueryFusionRetriever(top_k=10) -> AutoMergingRetriever(merge_threshold=0.5) -> SentenceTransformerRerank(model='BAAI/bge-reranker-base', top_n=3). Single function retrieve(query, index, docstore) -> results. Print each result: rank, score, text length (to show if parent was merged), first 300 chars.

Task 9	Generation — CitationQueryEngine + Groq LLM
Concept	The full RAG chain: query → retrieve top-3 chunks → build prompt with [SOURCE 1], [SOURCE 2] context → Groq LLM generates cited answer → CitationQueryEngine returns answer + source_nodes. This is the complete pipeline.
What to build	Write src/generation/rag_chain.py. Configure Groq LLM (llama-3.1-8b for voice, llama-3.3-70b for text). Build CitationQueryEngine over the retrieval pipeline. Ask 3 test questions from the RBI KYC document. Print cited answers.
Test data	Golden dataset question 1: 'What KYC documents does an NRI need for savings account opening?'
Verify it works	Answer contains [SOURCE 1] or [SOURCE 2] citations. response.source_nodes is not empty. Answer is factually correct per the RBI document. No hallucinated regulation numbers or penalty amounts.
Output	src/generation/rag_chain.py — ask(query, mode='voice' 'text') -> RAGResponse
Claude Code prompt	Write src/generation/rag_chain.py. Use Groq LLM via llama-index-llms-groq (model=llama-3.1-8b-instant). Build CitationQueryEngine with citation_chunk_size=150. System prompt: 'You are a fintech compliance assistant. Answer using ONLY the provided sources. Cite every factual claim with [SOURCE N].' Test questions: (1) What KYC documents does an NRI need? (2) What is the penalty for KYC non-compliance? (3) What is Enhanced Due Diligence? Print: answer text, citations used, source_nodes count.

9. Confirmed latency budget

With the confirmed stack (Groq Whisper STT + Groq Llama-3.1-8B + Cartesia Sonic TTS), the voice pipeline latency breaks down as follows. All values are streaming — the user hears the first spoken word at the sum of all steps, not after the full answer is generated.

Step	Component	Latency	Notes
1. STT	Groq Whisper-large-v3	~100ms	After user stops speaking. Groq LPU.
2. Intent routing	Keyword match (tier 1)	~30ms	No embedding call for voice — keyword rules only
3. BGE-M3 embed query	Local BGE-M3	~80ms	Query → 1024-dim vector, CPU
4. Weaviate + OpenSearch	Hybrid retrieval	~100ms	Dense + sparse, top-2 for voice
5. RRF + rerank	BGE cross-encoder	~60ms	Fast on top-4 candidates
6. Groq LLM first token	Llama-3.1-8B-Instant	~150ms	TTFT on Groq LPU, 750 t/s streaming
7. Cartesia TTS first word	Cartesia Sonic	~90ms	Byte-streaming from first LLM token
TOTAL to first spoken word		~610ms	Happy path, no guardrail blocks

Full answer streaming time by domain

Domain	Target full answer time	Binding factor
Customer support (voice)	~2-3 seconds	LLM generation length — short answers only
Regulatory Q&A; (voice)	~3-4 seconds	Context size (top-2 leaves) + answer length
Regulatory Q&A; (text)	~5-7 seconds	top-4 chunks + longer Sonnet-quality answer
Credit decisioning (text)	~6-10 seconds	Mixtral structured JSON generation
KYC/AML (text only, no voice)	~12-18 seconds	LangGraph 4-node workflow, Ollama local LLM

CAUTION

KYC/AML workflow is text-only — no voice interface. A 4-step stateful workflow taking 12-18 seconds is an unacceptable voice UX. Voice covers: regulatory Q&A; and customer support only.

10. Locked stack — complete reference

Layer	Tool / Library	Version	Free?	Decision
STT	Groq Whisper-large-v3	API	Yes (2k req/day)	ADR-002
TTS	Cartesia Sonic	API	Credits	ADR-003
LLM — voice + support	Groq Llama-3.1-8B-Instant	API	Yes	ADR-001
LLM — regulatory (text)	Groq Llama-3.3-70B	API	Yes	ADR-001
LLM — credit JSON	Groq Mixtral-8x7B	API	Yes	ADR-001
LLM — KYC/AML (PII)	Ollama Mistral-7B	Local	Yes	ADR-001
Voice orchestration	Pipecat	OSS	Yes	Phase 0
WebRTC transport	LiveKit (self-hosted OSS)	OSS	Yes	Phase 0
Mobile client	React Native + Expo + LiveKit RN SDK	OSS	Yes	ADR-006
Embeddings	BGE-M3 (BAAI)	HF Hub	Yes (MIT)	ADR-004
Vector store (dev)	Qdrant OSS	Docker	Yes (Apache 2.0)	Phase 2
Vector store (prod)	Weaviate OSS	Docker	Yes (BSD-3)	Phase 2
Keyword store	OpenSearch	Docker	Yes (Apache 2.0)	Phase 2
Reranker	BGE cross-encoder	HF Hub	Yes (MIT)	ADR-004
API framework	FastAPI	pip	Yes (MIT)	Phase 4
PII detection	Microsoft Presidio	pip	Yes (MIT)	Phase 5
Hallucination detection	DeBERTa NLI (HF)	HF Hub	Yes (MIT)	Phase 5
RAG evaluation	RAGAS + DeepEval + Promptfoo	pip/npm	Yes	ADR-005
STT evaluation	JIWER (WER)	pip	Yes (MIT)	ADR-005
LLM tracing	Langfuse (self-hosted)	Docker	Yes (MIT)	Phase 6
Metrics	Prometheus + Grafana	Docker	Yes (Apache 2.0)	Phase 6
Test data	RBI.org.in + SEBI.gov.in + BIS.org	Public PDFs	Free	All phases

12. Open items — decisions deferred to later phases

Item	Deferred to	Current assumption
Weaviate vs Qdrant for multi-tenant production	Phase 2	Qdrant for dev (simpler setup), Weaviate for multi-tenant prod
Exact chunk overlap percentage for sentence-aware splitting	Phase 1	10% overlap as starting point, tune after first eval run
Silero VAD threshold for financial domain (longer pauses between regulatory terms)	Phase 0	Default 0.5, tune during STT WER evaluation
Embedding model update strategy (BGE-M3 v2 when released)	Phase 2	Version-tag all embeddings, full re-embed on model upgrade
Groq rate limit backoff strategy under concurrent users	Phase 4	Exponential backoff with jitter, queue voice queries above text
Deepgram Nova-2 as STT upgrade for production (better VAD + financial fine-tune)	Phase 0 production	Groq Whisper for dev/demo, Deepgram for production if needed
Redis vs in-memory for rate limiting (Phase 4)	Phase 4	In-memory for dev, Redis for multi-pod deployment
Ollama model quantisation level (Q4 vs Q8 vs full precision)	Phase 1 / local setup	Q4_K_M as default — best balance of speed and quality on CPU

Living document: This ADR is updated at the end of each phase. v1.1 adds: ADR-007 (LlamaIndex orchestration) and the Phase 1 task breakdown with Claude Code prompts. v1.2 will cover Phase 1 results: chunking quality, retrieval evaluation scores, golden dataset baseline, and any decisions that changed during implementation. Version format: v{major}.{phase_number}.